

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2223/IIC2224 — Teoría de autómatas y lenguajes formales — 2' 2026

PAUTA TAREA 4

Pregunta 1

Considere el siguiente problema:

Problema:	#RUNS-NFA
Input:	Un NFA $\mathcal{A} = (Q, \Sigma, \Delta, I, F)$ y $w \in \Sigma^*$.
Output:	$\lvert\{\rho \mid \rho$ es una ejecución de aceptación de $\mathcal{A}$ sobre $w\}\rvert$.

Esto es, el problema #RUNS-NFA consiste en, dado un autómata finito no-determinista $\mathcal{A}$ y dado una palabra w, contar todas las ejecuciones de aceptación de $\mathcal{A}$ sobre w. Por ejemplo, usted puede comprobar que si $w \notin \mathcal{L}(\mathcal{A})$, entonces el output con $\mathcal{A}$ y w es 0.

Escriba una algoritmo que resuelva #RUNS-NFA en tiempo $\mathcal{O}(|\mathcal{A}| \cdot |w|)$ donde $|\mathcal{A}|$ es el número de estados y transiciones de $\mathcal{A}$. Demuestre la correctitud de su algoritmo.

Hint: Recuerda la evaluación de NFA "on-the-fly".

Solución

Una posible solución para esta pregunta es proponiendo la siguiente modificación al algoritmo *eval-NFAonthefly* visto en clases.

En lugar de llevar dos conjuntos S y S_{old} para llevar los estados en los que van las ejecuciones al leer la i-ésima letra de w, se llevará un **contador** por cada estado que acumula la **cantidad de ejecuciones** que hay en cada estado al leer la i-ésima letra. En el nuevo algoritmo, S y S_{old} corresponderán a arreglos de tamaño $|Q| = m$, donde cada entrada del arreglo le corresponde a un estado $j \in Q$, y esta llevará la cuenta de ejecuciones que han llegado a dicho estado j.

Sin pérdida de generalidad, suponga que el autómata $\mathcal{A}$ tiene como conjunto de estados $Q = \{0, 1, \ldots, m-1\}$. Si los m estados de $\mathcal{A}$ no son números, estos pueden ser ordenados arbitrariamente y ser asignados a números. También suponemos, sin pérdida de generalidad, que $\mathcal{A}$ es un autómata sin ϵ-transiciones. Luego, escribimos el siguiente algoritmo:

Algorithm 1 *

#RUNS-NFA($\mathcal{A} = (Q, \Sigma, \Delta, I, F)$, $w = a_1 a_2 \dots a_n$)

```
S, S_old ← Arreglos de largo m, y entradas con valor 0
for j = 0 to m − 1 do
    if j ∈ I then
        S[j] ← 1
    end if
end for
for i = 1 to n do
    S_old ← S
    S ← Arreglo de largo m, y entradas con valor 0
    for (j1, a_i, j2) ∈ Δ do
        S[j2] ← S[j2] + S_old[j1]
    end for
end for
c ← 0
for j ∈ F do
    c ← c + S[j]
end for
return c
```

Para la demostración de la correctitud del algoritmo, usaremos inducción sobre el tamaño de w, para mostrar que en toda iteración el arreglo S lleva correctamente la cantidad de ejecuciones que han llegado a cada estado.

CB: Antes de empezar a leer letras de w, se da que a los estados iniciales de $\mathcal{A}$ ha llegado una única ejecución, y todos los demás estados aún no les llega ninguna ejecución. Esto se ve representado correctamente al inicializar las entradas de S como 1 si le corresponden a un estado inicial, y 0 en caso contrario.

HI: Suponemos que luego de procesar $a_1 \cdots a_i$ se cumple que, para todo estado j, $S[j]$ es igual al número de ejecuciones de $\mathcal{A}$ sobre $a_1 \cdots a_i$ que llegan al estado j.

TI: Por HI, S lleva correctamente la cantidad de ejecuciones por estado en la i-ésima iteración. En la iteración $i+1$, se asigna a S_{old} los valores de S, y S se reinicializa con valores 0. Luego, por cada transición (j_1, a_{i+1}, j_2) del autómata, a $S[j_2]$ se le suma la cantidad de ejecuciones a j_1 al leer $a_1 \cdot a_i$. Al realizar esto sobre todas las transiciones del autómata, se cumple que $S[j]$ es igual al número de ejecuciones de $\mathcal{A}$ sobre $a_1 \cdots a_{i+1}$ para todo $j \in Q$.

Ya demostrado que el S lleva correctamente la cantidad de ejecuciones que llegan a cada estado para cualquier largo de w, la cantidad de ejecuciones de aceptación será la suma de la cantidad de ejecuciones que llegaron a estados finales del autómata después de leer las n letras de w. Esto se guarda en la variable c al final del algoritmo, y se retorna dicho valor. Luego, el algoritmo propuesto retorna correctamente la cantidad de ejecuciones de aceptación de $\mathcal{A}$ sobre w.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por usar algoritmo NFAonthefly buscando adaptarlo para la solución

- **(1 puntos)** Por usar contadores en los estados.
- **(1 puntos)** Por la actualización correcta en las transiciones de los contadores.
- **(1 puntos)** Por la inicialización de los estados iniciales y computo final en los estados finales.
- **(2 puntos)** Por demostrar la correctitud del algoritmo.

Pregunta 2

Sea Σ y Ω dos alfabetos finitos. Diremos que una función $h : \Sigma^* \to \Omega^*$ es un *homomorphismo* si para todo par de palabras $u, v \in \Sigma^*$ se cumple que $h(u \cdot v) = h(u) \cdot h(v)$.

Demuestre que para todo homomorphismo $h : \Sigma^* \to \Omega^*$ existe un transductor determinista $\mathcal{T}$ tal que $[\![\mathcal{T}]\!](w) = h(w)$ para toda palabra $w \in \Sigma^*$.

Solución

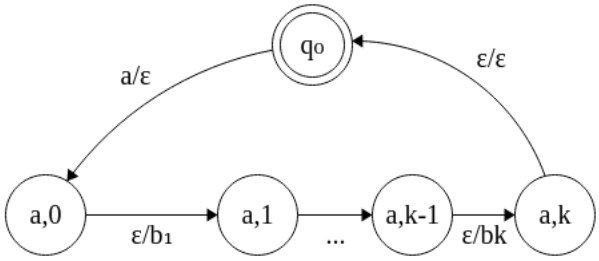
Sabemos que $h : \Sigma^* \to \Omega^*$ es homomorphismo. Por lo tanto, para cualquiera dos palabras $u, v \in \Sigma^*$, se tiene:

$$h(u \cdot v) = h(u) \cdot h(v)$$

Luego, podemos ver que para $w = a_1 \dots a_n$,

$$h(w) = h(a_1 \dots a_n) = h(a_1) \dots h(a_n).$$

También es fácil ver que $h(\varepsilon) = \varepsilon$, necesariamente. Por lo tanto, la función h queda definida por el resultado de $h(a) \in \Sigma^*$ para cada $a \in \Sigma$. Notar que el resultado de $h(a)$ es una palabra para cada $a \in \Sigma$. Entonces, intuitivamente podemos pensar en un transductor donde para cada a imprime la palabra $h(a) = b_1 \dots b_k$ con $b_i \in \Omega$ para todo $i \leq k$. Para esto, necesitamos un estado q_0 y, para cada $a \in \Sigma$, el transductor realiza la siguiente secuencia de transiciones:



Para definir esta secuencia formalmente, dado un $w = b_1 \dots b_k \in \Omega^*$, definimos $w[i] = b_i$ como la i-ésima letra de la palabra w, indexado desde 1. Luego, más formalmente, podemos definir el transductor $\mathcal{T} = (Q, \Sigma, \Omega, \Delta, q_0, \{q_0\})$, tal que:

- $Q = \{q_0\} \cup \bigcup_{a \in \Sigma} \{(a, i) \mid 0 \leq i \leq |h(a)|\}$

- $\Delta = \bigcup_{a\in\Sigma}\left(\left\{\left(q_0, a, \varepsilon, (a,0)\right)\right\}\cup\left\{\left((a,i{-}1), \varepsilon, h(a)[i], (a,i)\right) \mid 0 < i \leq |h(a)|\right\}\cup\left\{\left((a,|h(a)|), \varepsilon, \varepsilon, q_0\right)\right\}\right)$

En primer lugar, el transductor es determinista ya que para todo $(p,a,b,q),(p,a',b',q) \in \Delta$, se tiene que si $a = a'$ entonces $b = b'$. Por otro lado para todo $(p,\varepsilon,a,q),(p,\varepsilon,a',q') \in \Delta$, $a = a'$ y $q = q'$.

En segundo lugar, demostraremos que $\mathcal{T}(w) = h(w)$ para todo $w \in \Sigma^*$. Tomemos $w = a_1 \dots a_n$. El ejecutar el transductor sobre w tendremos una ejecución:

$$\rho : q_0 \xrightarrow{a_1/\varepsilon} (a_1,0) \xrightarrow{\varepsilon/b_{a_1,1}} (a_1,1) \xrightarrow{\varepsilon/b_{a_1,2}} (a_1,2) \xrightarrow{\varepsilon/b_{a_1,3}} \dots \xrightarrow{\varepsilon/b_{a_1,|h(a_1)|}} (a_1,|h(a_1)|) \xrightarrow{a_2/\varepsilon} (a_2,0)\dots q_0$$

Lo que da como output $b_{a_1,1}b_{a_1,2}b_{a_1,3}\dots b_{a_1,k_{a_1}}b_{a_2,1}\dots b_{a_n,k_{a_n}} = h(a_1)h(a_2)\dots h(a_n) = h(w)$.

Dado lo anterior, la distribución de puntaje es la siguiente:

- **(1 punto)** Por argumentar que la función h depende solamente de Σ.
- **(1 puntos)** Por construir el transductor $h(a)$ para $a \in \Sigma$.
- **(1 puntos)** Por la construcción de los estados Q.
- **(1 puntos)** Por la construcción de las transiciones en Δ.
- **(1 punto)** Por argumentar que el transductor es determinista.
- **(1 punto)** Por demostrar que $\mathcal{T}(w) = h(w)$.